Sports Betting Prediction & Parlay Intelligence App

Contractor Statements of Work

Data Scientist / Sports Quant Model Developer
Full-Stack / Backend AWS Developer

Prepared for POC contractor sourcing and evaluation

Document Purpose	Summary
Contractor-ready SOW packet	Use this document to source, evaluate, and contract a sports quant/data scientist and a full-stack/backend developer for the proof of concept.
Current state	The existing platform has early GitHub and AWS progress and appears to be pulling sportsbook odds data, but storage location and calculation application must be audited and repaired.
POC objective	Build one sport first, prove the data pipeline, individual predictions, three-leg parlay rankings, and historical accuracy dashboard, then expand sport by sport.

Recommended hiring sequence: Run the backend/full-stack audit and data scientist model design in parallel. The developer fixes the pipeline and builds the product; the data scientist defines the prediction and parlay logic.

Contents

SOW 1: Data Scientist / Sports Quant Model Developer

1. Project Overview

We are building a sports betting prediction and parlay intelligence app. The product must provide individual game winner predictions, confidence scores, market signal tags, ranked three-leg parlays, and historical accuracy reporting by sport and time period.

The company does not yet have complete algorithms built for every sport type. This engagement will focus on creating a reusable universal prediction framework and the first sport-specific model module, then documenting how future sports should be added.

2. Contractor Role

The contractor will serve as the Data Scientist / Sports Quant Model Developer. This person owns the intelligence layer of the product, including prediction logic, scoring methodology, signal definitions, parlay ranking, historical accuracy measurement, and developer-ready model documentation.

3. Key Principle: One Sport First

The POC should not attempt to build complete algorithms for every sport at once. The first version should prove the end-to-end system for one sport, preferably NBA or MLB, then expand using sport-specific modules.

Layer	Purpose	Examples
Universal Engine	Logic that applies across sports	Odds conversion, implied probability, vig adjustment, T1/T2/T3 movement, steam, resistance, compression, reversal, multi-book agreement, confidence score, prediction tracking, parlay ranking, historical accuracy
Sport-Specific Module	Logic that changes by sport	NBA injuries/rest/star availability; MLB pitchers/bullpen/weather; NFL injury reports; NHL goalie confirmation; soccer draw outcome; tennis surface/fatigue; NCAAM volatility

4. Core Responsibilities

- Define the first sport-specific prediction algorithm for the POC.
- Define a universal scoring framework that can support future sports.
- Define individual game prediction logic and confidence scoring.
- Define market signal tags such as steam, resistance, trap, coin flip, anchor, upset watch, compression, reversal, and multi-book disagreement.
- Define three-leg parlay ranking logic for all eight possible outcomes.
- Define historical accuracy calculations for individual picks and parlays.
- Create developer-ready formulas, pseudocode, or Python code.
- Create plain-English explanation templates for users.
- Work with the full-stack/backend developer to ensure the model can be implemented in the data pipeline.

5. Required Model Outputs

Output Area	Required Outputs
Individual Game Prediction	Predicted winner, confidence score, implied probability, market-adjusted probability, classification, signal tags, explanation
Three-Leg Parlay Ranking	All eight combinations, rank, legs, individual odds, parlay decimal odds, parlay American odds, implied win probability, underdog count, signal alignment score, confidence score, explanation
Historical Accuracy	Daily, rolling 7-day, rolling 21-day, and rolling 30-day accuracy by sport for individual picks and parlays
Backtesting	Prediction result, actual winner, rank #1 hit rate, Top-3 containment rate, Top-4 containment rate where applicable, performance by sport and signal type

6. Required Signal Framework

The data scientist must define the following signals, including definition, required data, scoring impact, and example usage:

- Steam move
- Resistance
- Favorite weakening
- Underdog strengthening
- Market compression
- Reverse line movement
- Trap risk
- Coin flip
- Anchor strength
- Upset pressure
- Multi-book confirmation
- Multi-book disagreement
- Late movement
- False stability
- High-variance game

7. Required Calculations

- American odds to decimal odds
- American odds to implied probability
- Vig-adjusted probability where possible
- Probability movement from T1 to T2, T2 to T3, and T1 to T3
- Favorite/underdog classification
- Underdog count
- Parlay decimal odds
- Parlay American odds
- Parlay implied probability
- Signal score
- Confidence score
- Historical accuracy formulas

8. Historical Accuracy Requirements

The model documentation must define how historical accuracy is calculated and reported for both individual picks and three-leg parlays.

Metric	Formula / Definition
Individual Pick Accuracy	Correct individual picks / total individual picks
Rank #1 Parlay Hit Rate	Number of parlays where the rank #1 combo won / total parlays
Top-3 Containment Rate	Number of parlays where the winning combo ranked 1-3 / total parlays
Sport Accuracy	Correct predictions for a sport / total predictions for that sport
Rolling Accuracy Windows	Daily, rolling 7-day, rolling 21-day, and rolling 30-day performance by sport

9. Deliverables

1. First sport-specific prediction methodology
2. Universal prediction and scoring framework
3. Confidence scoring methodology
4. Signal definitions and recommended weights
5. Three-leg parlay ranking methodology

6. Historical accuracy and backtesting methodology
7. Example calculations using sample odds data
8. Python code, formulas, or pseudocode
9. Data dictionary and developer integration notes
10. Plain-English explanation templates for the user interface

10. Milestones

Milestone	Deliverables	Estimated Timeline	Estimated Cost
1. Model Design	Select first sport, define required data, universal framework, sport-specific assumptions, and signal taxonomy	1 week	$3,000-$7,500
2. Game Prediction Logic	Predicted winner logic, confidence score, classification rules, example outputs	1-2 weeks	$5,000-$12,500
3. Parlay Ranking Logic	Eight-combination ranking method, parlay confidence score, underdog count logic, anchor/coin-flip/hedge rules	1-2 weeks	$7,500-$15,000
4. Historical Accuracy & Backtesting	Daily, 7-day, 21-day, and 30-day accuracy logic; rank #1 and Top-3 metrics; signal performance metrics	1-2 weeks	$5,000-$12,500
5. Developer Handoff	Python/pseudocode, formulas, data dictionary, integration notes, model summary	1 week	$3,000-$7,500

11. Estimated Total Cost and Timeline

Item	Estimate
Total cost range	$20,000-$50,000
Target budget	$25,000-$40,000
Timeline	4-8 weeks
Recommended structure	Part-time in parallel with backend/full-stack audit and build

12. Acceptance Criteria

- The first sport-specific algorithm is defined and documented.
- The universal scoring framework is documented for future sports.
- Individual game predictions can be produced from stored odds data.
- Confidence scores and signal tags are clearly defined.
- All eight three-leg parlay outcomes can be ranked.
- Historical accuracy formulas are documented for individual picks and parlays.
- Developer-ready formulas, code, or pseudocode are delivered.
- Plain-English explanation templates are included.

SOW 2: Full-Stack / Backend AWS Developer

1. Project Overview

We are building a sports betting prediction and parlay intelligence POC. The existing platform has early GitHub and AWS progress and appears to be pulling sportsbook odds data, but the current data path is unclear and calculations are not being applied to the pulled data.

This contractor will audit, repair, and extend the existing platform into a working POC that supports odds ingestion, snapshot storage, calculation application, individual game predictions, three-leg parlay rankings, historical accuracy tracking, and a user-facing dashboard.

2. Contractor Role

The contractor will serve as the Full-Stack / Backend AWS Developer. This person is the primary builder of the POC. The contractor must be able to work with an existing GitHub repository and AWS environment, audit what exists, repair issues, and build the application from the current foundation where practical.

3. Required Skills

Area	Required Skills
Backend / Cloud	AWS Lambda, API Gateway, CloudWatch, EventBridge, IAM, S3, Secrets Manager or Parameter Store
Database	DynamoDB or PostgreSQL, schema design, query design, indexing, result storage
Programming	Python or Node.js; TypeScript preferred where applicable
Frontend	React or Next.js, responsive dashboard, API integration
Data Pipeline	API ingestion, normalization, snapshot storage, calculation pipeline, error handling
Dev Workflow	GitHub, branches, pull requests, documentation, deployment process
Preferred Bonus	Sports betting, odds data, fintech, financial-market-data, or analytics dashboard experience

4. Existing Platform Audit

The first milestone must be an audit. The contractor must not begin major new feature development until the existing data flow is documented and repaired.

- Identify what function or service is pulling odds data.
- Identify what API or data source is being called.
- Determine whether the pulled data is being stored.
- Identify the storage destination, if any.
- Determine whether data is only being logged in CloudWatch.
- Identify missing environment variables, IAM permission issues, table-name issues, or silent failures.
- Determine why calculations are not being applied to pulled data.
- Document what can be preserved, refactored, removed, or rebuilt.

5. Required Data Pipeline

The backend must support the following flow:

```
fetchOdds() -> saveRawSnapshot() -> normalizeSnapshot() -> calculateImpliedProbabilities() -> compareSnapshots() -> applySignalRules() -> generateGamePredictions() -> generateParlayCombinations() -> rankParlays() -> saveModelOutputs() -> serveResultsToFrontend()
```

6. Required Storage Structure

Table / Collection	Purpose	Representative Fields
Raw Odds Snapshots	Store exact API response and source metadata	snapshot_id, sport, league, book, game_id, teams, commence_time, moneylines, total, timestamp, snapshot_label, raw_payload
Normalized Odds Snapshots	Store cleaned team-level odds used by the model	snapshot_id, game_id, book, team, opponent, moneyline, decimal_odds, implied_probability, vig_adjusted_probability, favorite_status, snapshot_label, timestamp

Market Signals	Store movement and signal outputs	game_id, team, book, T1/T2/T3 moneylines, implied probabilities, deltas, steam_flag, resistance_flag, trap_flag, coin_flip_flag, anchor_flag, upset_watch_flag, signal_score
Game Predictions	Store individual game predictions	game_id, predicted_winner, confidence_score, classification, signal_summary, created_at
Parlay Rankings	Store all ranked three-leg parlay outputs	slate_id, rank, leg teams, parlay odds, implied probability, underdog count, confidence_score, explanation, created_at
Historical Results	Store outcomes and accuracy tracking	actual_winner, prediction_correct, winning_combo_rank, rank_1_hit, top_3_containment, sport, date, settled_at

7. Required Backend Features

- Odds data ingestion from the selected odds source/API.
- Snapshot labeling for T1, T2, T3, and optional T4.
- Raw and normalized odds storage.
- Decimal odds conversion and American odds conversion.
- Implied probability and optional vig-adjusted probability calculation.
- Favorite/underdog identification.
- T1/T2/T3 movement calculations.
- Support for signal tags from data scientist logic.
- Individual game prediction storage and retrieval.
- Three-leg parlay generation with all eight outcomes.
- Parlay odds, implied probability, underdog count, confidence score, and rank storage.
- Historical result storage and accuracy aggregation.
- API endpoints for frontend display.
- Error logging, data validation, and admin/debug visibility.

8. Required Frontend / Dashboard Features

- Main dashboard with sport/date selector and today's games.
- Individual game prediction cards showing predicted winner, confidence score, classification, signal tags, and explanation.
- Game detail page showing sportsbook odds, T1/T2/T3 snapshots, movement, and signal explanation.
- Three-leg parlay builder allowing selection of three games.
- Ranked parlay output showing all eight combinations, rank, odds, implied probability, underdog count, confidence score, and explanation.
- Historical accuracy dashboard by sport and date range.
- Internal admin/testing view showing last data pull, storage status, missing fields, calculation status, and model output status.

9. Historical Accuracy Dashboard Requirement

The platform must show historical accuracy of each sport type broken down by day, rolling 1-week, rolling 3-week, and rolling 1-month periods for both individual picks and three-leg parlays.

Dashboard Metric	Required Breakdown
Individual Pick Accuracy	Sport, date, daily, rolling 7-day, rolling 21-day, rolling 30-day, confidence band, signal type
Parlay Rank #1 Hit Rate	Sport, date, daily, rolling 7-day, rolling 21-day, rolling 30-day
Parlay Top-3 Containment	Sport, date, daily, rolling 7-day, rolling 21-day, rolling 30-day
Parlay Top-4 Containment	Optional where applicable by sport/model variant
Performance Trends	Best/worst sport, accuracy trendline, signal-performance summary, individual vs parlay performance

10. Deliverables

1. Existing GitHub/AWS audit report

2. Data pipeline map and repair plan

3. Confirmed storage destination and corrected schema

4. Raw and normalized odds storage layers

5. Calculation pipeline for implied probability and movement deltas

6. Model output integration layer

7. Backend APIs

8. Frontend dashboard

9. Three-leg parlay builder and ranked outputs

10. Historical accuracy dashboard

11. Admin/debug view

12. Cloud deployment

13. GitHub documentation and handoff walkthrough

11. Milestones

Milestone	Deliverables	Estimated Timeline	Estimated Cost
1. Existing Codebase & AWS Audit	Audit GitHub/AWS, identify current data pull, storage destination, calculation failure points, repair plan	1 week	$3,000-$8,000
2. Data Storage Repair	Fix storage issue, save raw odds, save normalized odds, schema, logging, error handling	1-2 weeks	$5,000-$15,000
3. Calculation Pipeline	Decimal odds, implied probability, favorite/underdog classification, T1/T2/T3 deltas, signal support	1-2 weeks	$7,500-$17,500
4. Prediction & Parlay Backend	Prediction output storage, all eight parlay combinations, parlay odds, ranking storage, APIs	2-3 weeks	$10,000-$25,000
5. Frontend Dashboard	Game cards, prediction display, parlay builder, ranking pages, historical accuracy view, admin view	2-4 weeks	$15,000-$35,000
6. Deployment & Handoff	Deploy backend/frontend, QA, documentation, handoff walkthrough	1 week	$3,000-$8,000

12. Estimated Total Cost and Timeline

Build Option	Estimated Cost	Timeline	Notes
Lean Full-Stack POC	$50,000-$75,000	6-8 weeks	One senior full-stack/backend developer covers backend and basic frontend; data scientist separate
Strong Full-Stack POC	$75,000-$110,000	8-10 weeks	Cleaner UI, stronger dashboard, historical accuracy, admin/debug tooling
Investor-Ready MVP	$110,000-$175,000+	10-16+ weeks	Subscriptions, polished UI, multiple sports, stronger scale/security, advanced analytics

13. Acceptance Criteria

- The existing data pull is fully traced and documented.
- Raw odds data is stored correctly.
- Normalized odds data is stored correctly.
- Calculations are applied to pulled data.
- T1/T2/T3 movement can be calculated.
- Game predictions can be stored and retrieved.
- Three selected games generate all eight parlay outcomes.
- Ranked parlay outputs display correctly.
- Historical accuracy is displayed by sport, day, rolling 1-week, rolling 3-week, and rolling 1-month periods.
- Individual pick accuracy is separated from parlay ranking accuracy.

- The dashboard is deployed and demo-ready.
- GitHub documentation and handoff notes are complete.

Recommended 30-Day Execution Plan

Timeframe	Backend / Full-Stack Developer	Data Scientist / Sports Quant
Week 1	Audit GitHub/AWS, identify data pull, find storage destination, inspect CloudWatch/IAM/database issues	Select first sport, define required data, draft universal framework and sport-specific assumptions
Week 2	Repair storage, create/confirm raw and normalized tables, add logging and error handling	Define signal taxonomy, implied probability logic, confidence scoring, and classification rules
Week 3	Build calculation pipeline, compute odds conversions, T1/T2/T3 deltas, expose basic APIs	Create sample predictions and ranked parlay examples using sample data
Week 4	Store prediction/parlay outputs, start dashboard/API integration, add admin/debug status	Finalize backtesting and historical accuracy methodology, hand off formulas/code

Recommended Budget Ranges

Role	Recommended Engagement	Budget Range
Backend / Full-Stack Developer	25-35 hours/week for 6-8 weeks; starts with $3K-$8K audit	$50K-$75K lean; $75K-$110K strong
Data Scientist / Sports Quant	8-15 hours/week for 4-8 weeks; begins in parallel with audit	$25K-$40K target; $20K-$50K range
Optional UI/UX Designer	1-3 week design sprint if developer cannot produce clean demo UI	$5K-$15K

Candidate Screening Questions

For Full-Stack / Backend AWS Developer

- Have you audited an existing AWS/serverless application before? Describe the process.
- How would you determine where a Lambda function is sending or storing API data?
- How do you debug missing DynamoDB writes, IAM permission issues, and environment variable issues?
- Can you work inside an existing GitHub repo using branches and pull requests?
- Have you built dashboards that display model outputs or analytics data?
- Have you worked with odds data, financial-market data, or time-series snapshots?

For Data Scientist / Sports Quant

- Have you worked with sportsbook odds, moneylines, implied probability, or vig adjustment?
- How would you convert T1/T2/T3 odds movement into prediction signals?
- How would you separate a strong favorite from a coin flip or upset-watch game?
- How would you rank all eight outcomes of a three-leg parlay without simply ranking by odds?
- How would you measure whether the model is improving over time?
- Can you deliver developer-ready formulas, Python code, or pseudocode?